

Databázy

1. Sémantika dotazov.

- **Teórie, modely, stratifikovaná negácia, lokálne stratifikovaná negácia.**

- **Sémantika pevného bodu.**

- **Gelfond-Lifschitzova transformácia, stabilné modely, well-founded model.**

- **Teórie, modely, stratifikovaná negácia, lokálne stratifikovaná negácia.**

- Teorie, modely:

- Teoria je množina formul (axiom)

- Datalogový program je teória

- Implikácie (pravidla) sú axiomami

- V EDB sú iba pravdivé veci

- Preto je táto teória vždy konzistentná

- Jediným pravidlom je modus ponens

- Teda sa neda dokázať, že je niečo False

- Model teórie je také naplnenie relácií, že axiomy platia

- Minimalný model je taký, že žiadna jeho vlastná podmnožina nie je model

- Ak je program bez negácií, tak minimalný model je rovnaký ako fixpoint (z naivnej evaluácie)

- Ak je program s negáciami, tak nemusí mať fixpoint ($p :- \text{not } p$)

- Program s negáciou môže mať viac minimalných modelov

- Nas zaujímajú iba doménovo nezávislé formuly

- Stratifikovaná negácia, lokálne stratifikovaná negácia:

- Pre program vieme nakresliť dependency graph

- Ak je v tomto grafe cyklus s negovanou hranou, tak nie je stratifikovaný

- Lokálna stratifikácia: vrcholy nie sú iba predikáty ale všetky instancie predikátov podľa EDB

- Teda instancujeme program všetkými možnosťami a z toho nakreslíme dependency graph

- V lokálne stratifikovanom programe fixpoint = minimalný model

- **Sémantika pevného bodu.**

- Pevný bod je jednoznačný a existuje v:

- Programoch bez negácie

- Stratifikovaných programoch

- Lokálne stratifikovaných programoch

- Well-founded semantike

- **Gelfond-Lifschitzova transformácia, stabilné modely, well-founded model.**

- Aj keď program nie je lokálne stratifikovaný, možno vieme počítať odpoveď vzhľadom na iný ako minimalný model

- Gelfond-Lifschitzova transformácia, stabilné modely:

- Začneme s čiastočnou interpretáciou M (nejakou)

- Instancujeme pravidla všetkými možnosťami podľa EDB

- Zmazať PRAVIDLA ktoré obsahujú NOT atom kde atom patrí do M

- Zmazať PODCIELE ktoré obsahujú NOT atom kde atom nepatrí do M

- Zo zvyšku zrať fixpoint podľa EDB

- Tým dostaneme $GLT(M)$

- Ak $M = \text{GLT}(M)$ tak ide o stabilny model
- Stabilnych modelov moze byt viac (alebo aj ziadny)

- Well-founded model:

- 3 hodnotovy model (true, false, unknown)
- Budeme robit "alternating-fixpoint"
- Zacneme s tym, ze vsetky mozne instancie hlav pravidiel su False
- V iteracii spravime GLT, cim dostaneme nejake nove T/F. Ak je to novy stlpec tak opakujeme GLT
- Medzi parnymi iteraciami klesa pocet False, medzi neparnymi klesa pocet True
- Ak uz dostaneme stlpec ktory sme videli, o 2 iteracie spat, tak sa pozrieme na posledne 2 stlpce a podla toho urcime T/F/U

- Ak je nieco vo well founded modeli True/False, tak je to True/False aj v kazdom stabilnom modeli
- Ak nemame ziadne Unknown, tak existuje prave jeden stabilny model (co je tento)
- Pre odpovede na queries nie je dolezite ci je nieco False/Unknown

[*] Inflacny model: Ak zakazeme vo well-founded pocitani zmeny z True na False, tak dostaneme inflacny model

2. Optimalizácia rekurzívnych dotazov.

- **Datalogové programy s funkčnými symbolmi, unifikácia termov, naivná evaluácia, seminaivná evaluácia.**

- **Prolog, SLD rezolúcia, rule-goal-tree, rule-goal-graph, väzby medzi premennými (binding patterns).**

- **Magická transformácia, zovšeobecnená magická transformácia.**

- Datalogové programy s funkčnými symbolmi, unifikácia termov, naivná evaluácia, seminaivná evaluácia.

Datalogove programy s funkcnymi symbolmi:

- Funkcny symbol: urcuje strukturu dat. Napríklad `s`` v predikate `nat(s(X)) <- nat(X)`.
- Vieme ukladať napr. XML dokumenty funkcnymi symbolmi (tagy budu funkčne symboly)

Unifikacia termov:

- Term matching
 - Matchujeme term t s ground-termom s .
 - Rekurzivny zostup cez strukturu
- Metoda zalozena na manipulaci mnoziny rovníc:
 - Mame mnozinu rovníc, vzdy jednu vyberieme a spracujeme
 - Ak je to rovnica $X = t$, tak ak X occurs in t tak sa neda, inac substituujeme X za t vo vsetkych rovniciach
 - Prolog nerobi occurs check, Datalog ano
 - Ak je to rovnica $f(X_1, \dots, X_n) = f(t_1, \dots, t_n)$ tak pridame do mnoziny rovníc rovnice $X_1 = t_1, X_2 = t_2$ atd
- Ina metoda: Escalada-Ghallab (reprezentacia termov stromami alebo DAGmi)
 - Ocislujeme vsetky vrcholy DFS-kom (vsetky maju rozne na ziaciatku)
 - Potom ideme od korena v prvom terme a matchneme ho s korenem v druhom terme (dame im rovnake cislo, union-set)
 - Aj deti sa musia navzajom matchnut

Naivna evaluacia:

- Do relacnej algebry sme pridali operatory atov, vtoa

- Tie su aby sme napasovali argumenty predikativ (s funkcnymi symbolmi) na stlpce prislusnych relacii a potom

vysledok napasovali na hlavu

- atov proste zoberie $p(f(X, Y), g(Z), W)$ a skusi vsetky mozne trojice (a, b, c) z korespondujucej relacie P a

ked sa namatchuje $p(f(X, Y), g(Z), W)$ na $p(a, b, c)$ pomocou substitucie S , tak do vysledku sa prida $(S X, S Y, S Z, S W)$.

- vto je ze ked sa vypocita telo, tak ten vysledok sa musi namapovat na premenne v hlave.

- Zacneme s prazdnymi relaciami

- Kým sa relácie menia, tak iterujeme

- Seminaivna iteracia:

- V dalsej iteracii robime iba s tym co pribudlo v minulej (aby sme nerobili veci dvakrat)

- Ak chceme robiť aj programy s negáciou, tak potrebujeme antijoin

- Výsledok bude vtoa(atov() JOIN atov() JOIN ... atov() ANTIJOIN atov() ANTIJOIN atov() ...)

- Negovane podciele sme odantijoinovali

- Prolog, SLD rezolúcia, rule-goal-tree, rule-goal-graph, väzby medzi premennými (binding patterns).

- Prolog

- Deklarativny jazyk

- Založený na prvoradovej logike

- Komponenty prologu:

- Fakty (EDB predikaty)

- character(achililes, illiad).

- Termy

- Premenne a funkce symboly (struktury)

- Pravidla

- Prolog nepoužíva naivnú evaluáciu a well-founded semantiku ako Datalog

- Pri unifikaciji nerobi occurs-check

- findall, bagof (backtrack na volnych premennych), setof (bagof ale odstrani duplikaty a usorti)

- SLD rezolucia:

- Dokaz sa bezne konstruuje od axiom pouzivanim modus ponens

- SLD rezolucia je sposob dokazu odzadu

- Prolog začne zo zaveru a hlada pravidla ktore maju hlavu ktora sa da matchnut. Ak najde tak matchne a ide

dokazovat podciele pravidla.

- Teda algoritmus postupne počíta nejakú substitúciu. Prolog vždy vyberá podciele na spracovanie zľava doprava.

- 4-port model

- Kedze SLD voli zlava doprava, vie postupne generovat najdene odpovede

- Call, Exit, Redo, Fail

- cut: Zrusi backtrack na podcieloach nalavo od neho a tiez zrusi dalsie pravidla (rovnake)

- !

- green cut: Ze to funguje pri lubovolnej permutacii pravidiel

- red cut: Funguje to spravidne iba pri vhodnom poradi pravidiel

- Rule-goal-tree, rule-goal-graph:

- RGT:

- V koreni je query (napr. $\text{anc}(j, A)$)

- Deti korena su pravidla s ktorých hlavami sa vie koren matchnut.
- Deti tych deti su podciele tych pravidiel
- Teda po urovniciach sa striedaju "goal" vrcholy a "rule" vrcholy
- Da sa prechadzat a pocitat pomocou expand-rule a expand goal
 - Ked tu sa zmeni DFS na BFS tak dostaneme QRG (queue-based RGT expansion)
 - My ale nechceme nahradit naivnu evaluaciu tymto. Chceme ju upravit aby pocitala to iste.
 - Lebo naivna je ez, iba jeden cyklus, nie vzajomne rekurzivne funkcie

- RGG:

- Zovseobecnenie RGT
- Vrcholy maju ozdoby (goal vrcholy maju bff atd. a rule vrcholy maju zoznam bound a free premennych)
- Su tam vrcholy typu $r2.0[X, Y | Z]$
- Je to vzdy konecne (ale moze byt cyklicke)
- Preusporiadanie podcielov v pravidle moze zmensit RGG
 - Treba backtracking
- Feasibility problem pre RGG: moznost treba poprehadzovat podciele aby built-in predikaty davali zmysel
- Rektifikacia: p^{ff} pre $p(X, X)$ nie je uplne vystizne. Prepiseme to aby sme nemali duplikovane argumenty ani

konstanty

- Ak povodny bol bezpecny, tak aj novy bude (len sa nesmieme pytat na nove predikaty)

- Binding patterns:

- Bound, free premenne

TODO

- Magická transformácia, zovšeobecnená magická transformácia.

- Magická transformácia:

- Pre danu query transformuje program tak, ze zrata iba to co top-down pre tu query.
- Zachovava bezpecnost
- Zacneme z realizovatelneho RGG
- Pridame magicky predikat pre kazdy IDB predikat
- Pridame supplementary predikat pre kazdy rule node (teda pre $r2.1$ sa prida $sup2.1$)
 - Argumentami toho supu su bound premenne toho $r2.1$
 - Supplementary predikaty zodpovedaju tym relaciam pri bublani pravidlom zlava doprava
- Kazda hrana RGG sa prelozi na pravidlo magickeho programu
 - Bude niekoľko typov hran
 - Uvodna hrana (od query do korena RGG)
 - Spravi fakt pre magic predikat s tymi bound vecami.
 - Vsetky ostatne hrany co idu do IDB predikatov (rekurzivne volania)
 - Tie su definovane tym co do nich vchadza
 - Hrany vstupujuce do pravidla (teda sipky do r_i_0)
 - Tiez definovane tym co do nich vchadza
 - Hrany prechadzajuce cez pravidlo okrem poslednych (teda z r_i_j do $r_i_(j+1)$)
 - Definovane predchadzajucim sup predikatom a tym cez co presiel
 - Hrany ukoncujuce prechod pravidla (teda do posledneho r_i_j , teda kde j je maximalne)
 - Tieto definuju uz tie skutocne predikaty (nie magicke atd). Pouzije sa v tele posledny sup a DB volanie.
 - $sg(X, Y) \leftarrow sup_2.2(X, XP, YP), par(Y, YP).$

- S negaciou je to zlozitejsie

- Niekedy vieme namiesto magickej transformácie ručne expandovať podciele
- Zovšeobecnená magická transformácia
 - Dajú sa používať všetky typy viazaní
 - Duplikované argumenty
 - Konstanty
 - Netreba tu rektifikáciu a ozdoby (teda ozdoby treba iba na EDB / vstavane)
 - Výpočet takehoto programu je ťažký
 - Každému IDB predikátu sa dá magický predikat (s rovnakým počtom argumentov)
 - Každý sup predikat má len argumentov ako je #premenných v tele
 - Zvyšok je podobný ako pri inej magickej transformácii
 - Teda rozdiel je v podstate iba v tom že používame všetky argumenty a nie iba bound

3. Optimalizácia nerekurzívnych dotazov.

- **Strom relačného výrazu, hypergraf dotazu.**
- **Dekompozícia hypergrafu, Wong-Youssefiho algoritmus.**
- **Semijoin, úplný reduktor, Yannakakisov algoritmus.**
- **Optimalizácia poradia joinov.**
- **Odhad veľkosti výstupu dotazu.**
- **Použitie indexov, dekompozícia databázy.**
- **Pohltenie (containment) a ekvivalencia konjunktívnych dotazov.**
- **Testovanie pohltenia dotazov, petrifikované dotazy, kanonické databázy.**
- **Tablá, vynútenie závislostí, slabá ekvivalencia dotazov, minimalizácia tabiel.**

- Strom relačného výrazu, hypergraf dotazu.
 - Strom výrazu:
 - Vnútorný vrchol je operácia rel. algebry (π , σ , ...)
 - Listy sú relácie
 - Hypergraf
 - Hrana obsahuje niekoľko vrcholov (možno 1)
 - GYO redukcia hypergrafu
 - Ak je hrana podmnožinou inej, odstrániť tú menšiu hranu
 - Ak je vrchol iba v jednej hrane, odstrániť vrchol
 - Prázdne hrany zmiznú
 - Graf je GYO-redukovateľný, ak GYO redukciou zmizne
 - Taky sa vola acyklický
 - Trhanie úsi
 - Databázová obdoba GYO redukcie
 - Hrana E je ucho, ak je izolovaná, alebo ak existuje hrana F že všetky vrcholy z E - F sú iba v E
 - Algoritmus odstraňuje úsi
- Hypergraf dotazu:

- Premenne a konstanty su vrcholy
- Relacne hrany: pre relacie
 - Napriklad pri `select R.x from R, S` by sme mali hranu R (obsahujucu R.x, R.y) a hranu S
- Selekčne hrany (napríklad mensi ako), napr. hrana medzi R.y, S.y ak je `where R.y < S.y`
- Hrany na rovnosť neexistujú, veci čo sú rovné zúčime do 1 uzla (aj konstanty)

- Tymto dostaneme standardny hypergraf

- Dekompozícia hypergrafu, Wong-Youssefiho algoritmus.

- Budeme postupne z grafu odstraňovať hrany a počítat kód dvoch fáz

- Prvá fáza chce zredukovať relácie (prog)
- Druhá fáza ich joinuje (res)

- Rozlíšime 3 prípady

- Hypergraf je viac komponent
 - prog získame zlepením progov pre komponenty
 - res bude súčin resov pre komponenty
- Zmázeme selekčnú hranu, ostane H
 - $prog(G) = prog(H)$
 - $res(G)$: získame selekciou podľa tej odstranenej hrany na $res(H)$
- Zmázeme relacnú hranu, ostane komponenty $H_1, \dots, H_k$

- Každú relacnú hranu čo sa pretínala s odstranenou zredukujeme podľa odstranenej ($S := S \text{ SEMIJOIN } R$) a zlepieme

progy pre komponenty. Cize forall $S: \{S := S \times R\}; prog(H_1); \dots; prog(H_k);$

- $res(G) := R \text{ JOIN } res(H_1) \text{ JOIN } \dots \text{ JOIN } res(H_k)$

- Preferujeme odstraňovanie relacných hran (to môžeme ak ju nepretína selekčná a ak má prienik aspoň s nejakou inou relacnou)

- Preferujeme odstraňovanie malých relacných hran (že majú konstantu alebo už boli redukované SEMIJOIN-om)
- Preferujeme rozbíjanie na komponenty

- Semijoin, úplný reduktor, Yannakakisov algoritmus.

- Semijoin:

- Opak antijoinu. $R \text{ SEMIJOIN } S$ vráti tie riadky R ktoré sa joinuju s niečim z S
- Je to "filter" S nad R

- Úplný reduktor

- Postupnosť SEMIJOINov (a selekcií) sa volá reduktor
- Služí na redukciu n-tic čo vchádzajú do výpočtu
- Úplný reduktor je taký čo zabezpečí že do výpočtu nejde žiadna zbytočná n-tica
- Výpočet: Najdeme ucho E so svedkom F, zredukujeme F podľa E, rekurzívne zavoláme na hypergraf bez E,

zredukujeme zrkadlovo E podľa F

- Yannakakisov algoritmus

- Funguje iba pre acyklické hypergrafy, je rozšírenie Wong-Youseffiho
- V acyklickom hypergrafe vieme vždy najst úplný reduktor odtrhavaním usí
- Vstup je $PI_X(R_1 \text{ JOIN } R_2 \text{ JOIN } \dots \text{ JOIN } R_k)$
- Výstup je hodnota toho výrazu
 - Je tam tá projekcia, takže nechceme asi celý ten JOIN
- Najskôr zredukujeme $R_1, \dots, R_k$ pomocou úplného reduktora
- Potom zostavíme dekompozičný strom toho hypergrafu a prechádzame ho zdola hore a projektujeme tam niečo

cestou

- Optimalizácia poradia joinov.
 - Chceme mať čo najmenšie medzivýsledky
 - Môžeme prehľadávať všetky left-deep join stromy
 - To sa dá DP v $O(2^n)$
- Odhad veľkosti výstupu dotazu.
 - Pre $R \text{ JOIN } S$ môže byť výsledok medzi 0 a $|R||S|$
 - Optimalizátor si udržiava `katalog` kde si počíta statistiky pre odhad veľkosti výstupu
 - $B(R)$: počet blokov
 - $T(R)$: počet tuplov
 - $V(R, A)$: Kolko rôznych hodnôt má atribút A v R
 - $MIN(R, A)$, ...
 - Reduction factor hovorí ako sa zmenší výsledok oproti kartezskému súčinu
 - Ten sa ráta z veľkosti výsledku, ale ten nepoznáme. Ale budeme odhadovať RF
 - $rf(Q) = rf(AttrList) * rf(Cond)$
 - robíme so `SELECT AttrList From ??? WHERE Cond`
 - V podmienkach `and` je $*$, `or` je $+$ atď
 - Takto sa to naozaj počíta (postgres)
- Použitie indexov, dekompozícia databázy.
 - Indexy
 - Tie sa stavajú podľa toho ako sa tabuľka bude používať
 - Clustered index
 - Môže byť iba jeden pre tabuľku
 - Je podľa toho ako je fyzicky usporiadaná (väčšinou je to PK)
 - Chceme ho použiť na zgrupovanie rovnakých vecí, nemá význam keď je každej veci iba jedna
 - Non-clustered index
 - Môže ich byť veľa (hash, B-tree)
 - Ak sú hodnoty z malej domény (napr. známky) tak na ne nechceme index, lebo seq. scan možno bude rýchlejší keď je výsledok viac ako 20% relácie
 - Index cover: Môžeme spraviť index nad všetkými atribútmi tabuľky a potom query bude mať všetko z indexu bez prístupu do tabuľky
 - Pomocou B+ indexu to vieme spraviť "clustered"
 - Vnorené selecty sa optimalizujú samostatne, takže index na vnútornej tabuľke nepomôže kvôli niečomu z vonkajšej query
 - Na foreign keys by sme mali mať index, lebo zmazanie toho na čo ukazuje spustí integrity check v tabuľke ktorá má foreign key
 - Dekompozícia databázy
 - Denormalizácia môže zvýšiť výkonnosť castých queries
 - Asi zničí normálnu formu
 - Často znamená pridať redundantný atribút do tabuľky (taký čo sa inak musí zrať, ale dlho to trvá)
 - Nemali by sme používať denormalizáciu ak dobre nevieme čo robíme
 - Horizontal partitioning
 - Ak sa často pýtame na konkrétnu známku, tak môžeme rozdeliť tabuľku na 5 tabuliek podľa známok
 - Toto nerobíme manuálne, ale stroj to niekedy robí
 - Vertical partitioning

- Ak niektore atributy sa moc nepouzivaju, mozeme rozdelit tabulku na viac tabuliek
 - Tiez to stroje robia interne niekedy
 - Pohltenie (containment) a ekvivalencia konjunktívnych dotazov.
 - Konjunktívny dotaz:
 - Jedno Datalog pravidlo kde podciele su z EDB
 - Dotaz Q1 je pohlteny Q2 ak pre kazdu databazu je vysledok Q1 podmnozinou vysledku pre Q2
 - Konjunktívne dotazy su ekvivalentne ked sa navzajom pohlcuju
 - Moze sa to hodit pri optimalizacii
 - Pohltenie je NP-uplne, ale dotazy su zvycajne male
 - Testovanie pohltenia dotazov, petrifikované dotazy, kanonické databázy.
 - Testovat sa da dvomi sposobmi
 - Pohlcujuce zobrazenia
 - Zobrazenie z vacsieho Q2 do pohlteneho Q1. V podcieloch mapuje premenne.
 - Je dobre zacat hlavami
 - Zobrazenie nemusí pokryt vsetky podciele cieloveho (maleho) dotazu, iba musí existovat
 - Možno treba backtrackovať homomorfizmy, čo možno nechceme
 - Petrifikované konjunktívne dotazy
 - Petrifikovaný dotaz je databaza
 - Dostaneme ju tak, ze z premennych spravime "premenne-konstanty" (petrifikacia)
 - Pre kazdy predikat dame relaciu do vysledku
 - Do vyslednej tabulky pre predikat dame n-ticu "premennych-konstant"
 - $p(x, y) \leftarrow r(x, z), g(z, z), r(z, y)$ sa zmeni na $\{ r = \{(xp, zp), (zp, yp)\}, g = \{(zp, zp)\} \}$
 - Kanonicke databazy
 - Alternativa ku pohlcujucim zobrazeniam
 - Myslienka: Na vhodne narafoenej databaze ukazeme ze vysledok Q1 je pod Q2. Potom to bude platit aj pre kazdu inu databazu.
 - Narafoime petrifikovany dotaz ako databazu: $\text{petr}(Q1)$, kde Q1 je ten čo bude pohlteny
 - Vieme testovat aj ci je Q pohlteny programom P (spetrifikujeme Q a zbehneme aj Q aj P na tom a porovname vysledky)
 - Testovat vacsie veci je tazke (ci sa programy pohlcuju je nerozhodnutelne)
 - Co ked povolime negaciu?
 - Levy-Sagivov test
 - Na testovanie pohltenia
 - Spravi niekoľko kanonických databaz a ak to jedna pokazi tak ten dotaz nepohlcuje ten druhy
 - Skusi vsetky mozne databazy s domenou 0, 1, ... n kde n je pocet premennych v dotaze
 - Ked povolime aj predikat $\leq$:
 - Gupta-Zhang-Ozsoyogluov test
- Tablá, vynútenie závislostí, slabá ekvivalencia dotazov, minimalizácia tabiel.
 - Tablo:
 - Obdlznikovite pole
 - Stĺpce zodpovedaju atributom (zapisane v 1. riadku - hlavicka)
 - Druhy riadok je sumar: obsahuje hlavu dotazu
 - Premenne v tomto riadku volame "vznamne"
 - Riadky tabla zodpovedaju podcielom dotazu

- Vynutenie závislosti:
 - V tabľach vieme vynucovať FD, MVD, ...
 - Ak máme závislosť $A \rightarrow B$ tak ak sa dva riadky zhodujú na A, tak sa musia zhodovať aj na B (stotožníme ich atribúty B)
- Slabá ekvivalencia dotazov:
 - Slabé pohltenie: Ako pohltenie, ale nemusí platiť pre každú databázu ale len pre databázy čo sa spájajú bezstratovo
 - Slabá ekvivalencia dotazov: Navzájom sa slabo pohlčujú
- Minimalizácia tabiel:
 - Ak riadok r_1 v table pohlčuje riadok r_2 tak ten r_2 môžeme skrátiť

4. Vyššie normálne formy.

- Funkčné závislosti, 3NF, BCNF.

- Multizávislosti, 4NF.

- Joinovacie závislosti, 5NF.

- Inklúzne závislosti.

- DKNF (domain-key normal form).

- Metodika normalizácie.

- Funkčné závislosti, 3NF, BCNF.

- Funkčná závislosť $A \rightarrow B$ znamená že ak sa riadky tabuľky zhodujú na atribútoch A tak sa zhodujú aj na B
- 3NF:
 - Pre každú FD $X \rightarrow Y$ platí že buď X je nadkľúč, alebo Y je časťou nejakého kľúča
 - Každá binárna relácia je v 3NF
 - Testovanie či je 3NF je NP-tažké
 - Ale rozložiť do 3NF so zachovaním FD vieme polynomiálne
 - Algoritmus: Zlúč schému pre FD s rovnakou ľavou stranou, ak je schéma podmnožinou inej tak ju zmaz, ak žiadna podschéma neobsahuje kľúč, pridaj schému s kľúčom

- BCNF:

- Pre každú FD $X \rightarrow Y$ platí že X je nadkľúč
- Podobne ako 3NF veci tu platia, až na to že ten polý algoritmus na rozklad nezachováva FD
- Schéma v BCNF neobsahuje redundanciu vzhľadom na FD
- Multizávislosti, 4NF.
 - MVD (multi-valued-dependency)
 - $X \twoheadrightarrow Y$ hovorí že keď máme $r(X, Y_1, Z_1)$ a $r(X, Y_2, Z_2)$ tak máme určite aj s vymenenými Z, teda $r(X, Y_1, Z_2)$, $r(X, Y_2, Z_1)$
 - Platí tu nejaké rozšírenie Armstrongových axiom
 - 4NF vzhľadom na závislosti F (FD + MVD) je keď pre každú MVD $X \twoheadrightarrow Y$ z F^+ je X nadkľúč v r
 - r_1, r_2 tvoria `bezstratovú join dekompozíciu` vzhľadom na F ak:
 - $r_1 \cap r_2 \twoheadrightarrow (r_1 - r_2)$ alebo naopak
 - Algoritmus dekompozície do 4NF
 - naivný, ale garantuje bezstratovú join dekompozíciu

- Podobne ako BCNF dekompozicia
- Vždy najde reláciu S ktorá nie je v 4NF a tam najde MVD $X \twoheadrightarrow Y$ ktorá láme 4NF a nahradi S dvoma reláciami (S - Y) a (X zjednotenie Y)

- Nikdy nechceme lámať MVD pri 4NF dekompozícii, lebo potom môžeme mať priveľa redundancie

- Teda niekedy je lepšie nedeekomponovať keď sa neda
- Trik ako to obísť je pridať atribút (napríklad v Slovari atribút koncept)
 - Toto sa často používa pri 4NF dekompozícii
- Existuje nejaký komplikovaný algoritmus na 4NF dekompozíciu ktorý neláme MVD
 - Ak treba pridať atribút aby sa to dalo, tak ho prida

- Joinovacie závislosti, 5NF.

- Joinovacia závislosť:

- Zovšeobecnenie bezstratovosti

- Joinovacia závislosť je tvaru $R = R_1 \text{ JOIN } R_2 \text{ JOIN } \dots \text{ JOIN } R_n$ pre ktorú platí $r = \pi_{R_1}(r) \text{ JOIN } \pi_{R_2}(r) \text{ JOIN } \dots \text{ JOIN } \pi_{R_n}(r)$ bez ohľadu na naplnenie r

- R, R₁, ..., R_n sú množiny atribútov
- Oproti bezstratovosti je rozdiel že tu je to iba o atribútoch, nezávisia na konkrétnej dekompozícii
 - Teda tu R nemusí byť celá relácia

- MVD je špeciálnym prípadom joinovacej závislosti

- $R = (XY) \text{ JOIN } (X \text{ zjednotenie } (R - Y))$

- Pre tieto závislosti nie sú známe také pravidlá ako boli Armstrongove axiomy

- 5NF vzhľadom na F (FD, MVD, JD) je keď pre každú joinovaciu závislosť $JD(R_1, \dots, R_n)$ z F^+ platí že každé R_i je nadklúč v r

- Algoritmus na rozklad je dosť zložitý

- Inklúzne závislosti.

- Hovorí o dvoch reláciach

- $r.A < s.B$ ak pre každé naplnenie r, s platí že $\pi_A(r)$ je nadmnožina $\pi_B(s)$

- Teda že keď atribúty A v r nadobudajú nejakú hodnotu, tak aj v s musí byť táka hodnota (uložená v atribútoch B)

- Napríklad foreign key je inklúzna závislosť (ak je studentId v tabuľke grades tak je aj v tabuľke students)

- Majú inference rules (R1 - R3)

- DKNF (domain-key normal form).

- Najvyššia NF, je to len veľmi teoretické

- Fagin: Relácia je v DKNF ak každé obmedzenie je logickým dôsledkom kľúčov a domen

- Domain constraint

- Množina hodnôt ktorú môže atribút nadobúdať (farba môže byť {red, green, blue})

- Key constraint

- Hovorí že množina atribútov je nadklúč v relácii o ktorej hovoríme. Čiže hovorí o nejakej FD $K \rightarrow R$ kde K je

podmnožina atribútov relácie R

- Definícia:

- Nech D je množina domenových obmedzení, K množina kľúčových obmedzení a G množina všeobecných obmedzení (business rules) v R.

- Potom R je v DKNF ak $(D \text{ zjednotenie } K) \Rightarrow G$

- Nie je navod ako dekomponovať do DKNF

- Neda sa ani overiť či je niečo v DKNF

- Neobsahuje žiadnu redundanciu

- Metodika normalizácie.
- TODO

5. Distribuované databázy.

- Model distribuovanej databázy.
- Atomický commit. Požiadavky na ACP.
- Dvojfázový ACP. Trojfázový ACP. Výber koordinátora, bully algoritmus.
- Replikácia dát, distribuované zamykanie, distribuované deadlocky. Quorum protokol.
- Synchronizácia fyzických hodín, Christianov protokol, Berkeley protokol.
- Časové pečiatky, Lamportov protokol.

- Model distribuovanej databázy.

- Centralizovaná databáza iba číta stream operácií a rozhoduje sa čo robiť podľa vnútorného stavu
 - Vyhody: jednoduchosť, otestovanosť
 - Nevýhody: ťažko sa škáluje, nedostupnosť pri výpadku servera
- Požiadavky:
 - ACID, transparentnosť, odolnosť voči výpadku jedného servera
- Budeme mať niekoľko sites, na každej bude centralizovaný systém
 - Nad každým je dodatočná SW vrstva - site manager
 - Site manageri vedia navzájom komunikovať (keď chce site niečo čítať z iného situ napríklad)
 - Pre transakcie je load balancer ktorý ich priradzuje do síťov (1 tx vždy komunikuje len s 1 síťou keď sa prideli)
 - Čo keď chce niekto commitnúť? Musia s tým nejak všetky sites súhlasiť.
- Atomický commit. Požiadavky na ACP.
 - Bud musia všetky sites commitovať alebo všetky abortovať
 - Pri výpadku servera nám spoľahlivá sieť dá vedieť že ten server vypadol
 - Požiadavky na ACP:
 - [všetci rovnako] Všetky sites ktoré sa rozhodnú sa rozhodnú rovnako
 - [neprerozhodne sa] Ak sa site rozhodne, nikdy nezmení rozhodnutie
 - [commit iba ak yes] Rozhodnutie pre COMMIT sa môže iba keď všetci hlasujú YES
 - [ak všetci yes tak všetci commit] Ak všetci hlasujú YES a nie sú tam zlyhania, tak eventuelne sa všetky sites musia rozhodnúť pre COMMIT
 - [fungujúce sa rozhodujú] Ak dosť dlho nie sú zlyhania, tak fungujúce sites eventuelne spravia rozhodnutie
- Dvojfázový ACP. Trojfázový ACP. Výber koordinátora, bully algoritmus.
 - Dvojfázový ACP:
 - Home site bude koordinátor
 - Keď koordinátor sa rozhodne pre YES, tak spustí protokol
 - Najskôr všetkým ostatným síťom (participants) pošle správu "Vote!"
 - Participanti potom posielajú koordinátorovi svoje hlasy
 - Ak všetci poslali commit, tak koordinátor pošle správu "Global commit!" aby všetci commitli
 - Participanti ktorí commitli potom pošlú koordinátorovi správu "Acknowledge"
 - To je na to, aby mohol koordinátor zahodiť časť logu týkajúcu sa tejto transakcie

- Toto nesplna poslednu poziadavku na ACP, ked vypadne koordinador (participant mozu iba cakat kym sa zobudi)
- Ked sa participant zotavuje, pozrie do logu a spravi nieco podla COMMIT / ABORT, NO / YES
 - Ak sa zotavuje koordinador tak tiez podla logu

- Trojfazovy ACP:

- Koordnator posle "Vote!"
- Participant mozu svoje hlasy coordinatorovi
- Ak vseti chcu commitovat tak coordinator posle "Pre-commit"
- Vsetci mozu coordinatorovi acknowledge
- Koordnator posle vsetkym "Commit!"
- Ak koordinador vypadne pred pre-commitom, tak asi mozu vseti abortovat
 - Ak po pre-commite, tak si participant zvolia noveho koordinatora
 - Ten zozbiera rozhodnutia od ostatnych (rozhodnutia co posielal povodny koordinador)
 - Ak vsetci maju rozhodnutie, mozu ho spravit
 - Ak niekto ma pre-commit, tak novy koordinador spusti znova protokol od druhej fazy (posle pre-commit vsetkym)
 - Ked sa povodny koordinador zobudi, tak sa spyta ako to dopadlo a to spravi
- Toto este nefunguje ked mozu vypadavat aj linky (mozno sa vypadkom niekto odizoluje a ma pre-commit a nevidi susedov, tak commitne, ale susedia abortnu)

- Majority protocol:

- Skupina sitov moze commitnut iba ak ich je vacsina
- Niekedy to moze blokovat (ked musi)
- Este to nestaci, lebo ked vypadne novy koordinador a ina site s pre-commitom sa zobudi, tak nova majorita commituje, ale stary koordinador abortoval
- Pridame teda pre-abort
 - Ale co ked je v majorite rovnako vela pre-commitov ako pre-abortov?
 - Sites si pamataju ci naposledy robili abort alebo commit a podla toho sa spravi tie-break
 - Toto realne nikto neimplementuje (tie tie-breaks)

- Vyber koordinatora (bully algoritmus):

- Kazda site ma svoje id a kazda vie id ostatnych
- Chceme vybrat fungujucu site s max id a dat o tom vediet ostatnym
- Kazda node posielala "Election" spravy vsetkym (nie iba susedom) s vacsimi id. Ak naspat dostane "OK" tak vie ze sama nebude koordinador a caka
 - Spravy sa broadcastuju

- Replikácia dát, distribuované zamykanie, distribuované deadlocky. Quorum protokol.

- Replikacia:

- Data su replikovane na viacerych sites
- Z beznych technik sa daju spravit distribuovane (distributed locking, distributed casove peciatky, ...)

- Distribuovane zamykanie:

- Moze sa spravit centralizovane (jedna site manazuje vsetky locky), ale moze vypadnut manazer
- Moze sa spravit "primary copy" technika, ale ta je rovnako zla ako centralizovana
 - write lock manazuje site ktory ma primary copy dat
 - read lock maju vsetky sites ktore maju tie data
- Distributed 2-phase ROWA (read-one-write-all)
 - Read lock dava kazdy
 - Write lock musia pridelit vsetci spolu ktori maju tie data (teda drahy write)

- Majority 2-phase protokol
 - Aj read aj write lock sa musia ziskat od majority sitov ktore maju tie data
 - Moze nastat deadlock (3 txs chcu write lock, kazda ho ziskala u 1/3 sites)
- Quorum protokol:
 - Kompromis medzi ROWA a majority 2-phase protokolom
 - Kazda site dostane vahu
 - Read quorum Q_r a write quorum Q_w (mozu byt rozne pre rozne datove objekty)
 - Chceme aby $Q_r + Q_w \geq$ sucet vsetkych vah, aby bol dostatočný prekryv medzi citanymi a pisanymi, aby sme citali vzdy nieco aktualne
 - Read lock mozeme ziskat od sites ktore maju spolu sucet aspon Q_r
 - Podobne write lock
- Distribuovane deadlockly:
 - Riesenie: obcas sa deteguju deadlockly a abortuju podla toho txs
 - Kazda site si udrzuje WFG (wait-for-graph)
 - Obcas sa zbehne protokol na kombinovanie vsetkych WFG to globalneho a podla toho sa poabortuje
 - Globalny WFG sa spravi v jednej site nazývanej koordinátor
 - Koordinátor požiada o lokalne WFG
 - Skombinuje ich a abortuje pomocou ACP
 - Globalny WFG je kvoli oneskoreniam neaktualny
 - Teda niekedy sa abortuje zbytocne (niektore cykly uz neexistuju)
- Synchronizácia fyzických hodín, Cristianov protokol, Berkeley protokol.
 - Cristianov protokol
 - Na synchronizáciu hodin, velmi jednoduchy
 - Klient posle request na cas v case T_0 , dostane odpoved X v case T_1 a nastavi svoj cas na $X + (T_1 - T_0)/2$
 - Funguje dobre ked je RTT maly
 - Berkeley protokol
 - Vyberie sa jeden node - koordinátor
 - Ostatnych sa vzdy spyta aky si myslia ze je cas (aj seba). Na to pouziva Cristianov protokol.
 - Vypocita priemernu zmenu oproti svojmu casu a svoje hodiny tak nastavi a posle tuto informáciu o case vsetkym
- Časové pečiatky, Lamportov protokol.
 - Lamportov protokol je pre vzajomne vylucovanie v distrib. systemoch
 - Pouziva casove peciatky pozíadiaviek do kritickej sekcie
 - Request s najmensim timestampom ma prednost
 - Site moze poslat 3 typy sprav: REQUEST, REPLY, RELEASE
 - Kazda site ma svoju request_queue
 - Ked chce site cislo i vstupit do krit. sekcie, posle vsetkym REQUEST(ts_i, i)
 - Ked ina site S_j dostane takyto request, prida si i do svojej request_queue a posle naspat REPLY(ts_j, j)
 - Vykonat kriticku sekciu moze site i ak dostal od vsetkych REPLY s vacsim timestampom, ako je ten jeho ktory requestoval
 - Ked site skonci kriticku sekciu, posle RELEASE vsetkym a odstrani svoj request zo svojej queue
 - Ostatni si tiez tymto odstrania ten request z queue
 - Nevyhody:
 - Posiela sa vela sprav
 - Ked spadne nejaka site, tak sa to cele zastavi

- Vyhody:

- Jednoduchost
- Ferovost
- Skalovatelnost